

DetailFieldMap.json — Formatting Reference

This file controls what appears in the right-hand detail pane of the Project Navigator when a tree node is selected. It lives at:

```
Resources/DetailFieldMap.json
```

Edit it, save, and reload the navigator (or call `DetailHtmlRenderer.ReloadFieldMap()`) to see changes — no rebuild required.

1. Top-level structure

One entry per tree-node **tag type name** (the C# class name backing that node, e.g.

`HdwItemTag`, `POLineItemTag`, `DrawingSeriesSheetTag`):

```
json
{
  "SomeTag": {
    "title": "Display Title",
    "query": { ... },          // optional - see section 4
    "fields": [ ... ]
  }
}
```

- `title` (optional): heading shown at the top of the pane. Supports `{PropertyName}` placeholders substituted with the actual value, e.g. `"Sheet {SheetNumber}"` → `"Sheet 01"`. If omitted, falls back to an auto-generated name from the type (e.g. `DashHardwareItemTag` → `"Dash Hardware Item"`).
 - Types with no entry in this file still work — every public property on the tag is shown with an auto-generated caption.
-

2. Basic field entry

```
json
{ "key": "HdwNo", "caption": "Hardware Number", "order": 1 }
```

Property	Required	Description
<code>key</code>	yes	Must match a public property name on the tag class (case-sensitive), or a column returned by <code>query</code> (see section 4).
<code>caption</code>	yes	Natural-language label shown on the left.
<code>order</code>	yes	Controls row position, ascending. Decimals allowed (e.g. <code>9.5</code>) to slot between existing rows without renumbering.
<code>format</code>	no	.NET composite format string, e.g. <code>"{0:MM/dd/yyyy}"</code> , <code>"{0:N2}"</code> , <code>"{0:C}"</code> , <code>"{0:0.###}"</code> .

Notes:

- Empty/null values render as italic (**none**).
- If `key` doesn't exist on the tag or in query results, the row is silently skipped.
- Dates default to `MM/dd/yyyy` if no `format` given; booleans render as Yes/No.

3. Field types: text, link, image

Text (default)

```
json
```

```
{ "key": "HdwDesc", "caption": "Description", "order": 2 }
```

Link

```
json
```

```
{
  "key": "HdwVendorlink",
  "caption": "Vendor Link",
  "order": 11,
  "type": "link",
  "linkText": "Open Vendor Page"
}
```

- `linkText` (optional): display text instead of the raw URL. Supports `{OtherFieldKey}` placeholders, e.g. `"Open {HdwNo} Cutsheet"`.

- Links open in the user's **default browser** (not in the detail pane).
- Empty value → **(none)**.

Image

```
json
{
  "key": "HdwImage",
  "caption": "Image",
  "order": 13,
  "type": "image",
  "imageMimeType": "image/png",
  "maxWidth": 300,
  "maxHeight": 300
}
```

- Works with:
 - `byte[]` (SQL `image`/`varbinary` columns) — base64-encoded as a `data:` URI. Set `imageMimeType` to match how the image was stored (`image/png`, `image/jpeg`, etc.). Defaults to `image/png`.
 - A string URL (`http://`, `https://`, `file://`, `data:`) — used as-is.
 - A local or UNC file path — converted to a `file://` URI automatically.
 - `maxWidth` / `maxHeight` (optional, pixels) cap the displayed size while preserving aspect ratio.
 - Empty/unrecognized value → **(none)**.
-

4. Pulling extra fields from SQL (`query`)

If the tag object doesn't carry every field you want to show, define a `query` block to look up additional columns by key:

```
json
```

```

"HdwItemTag": {
  "title": "Hardware Item",
  "query": {
    "table": "dbo.Proj_HdwCompile",
    "keyColumn": "ID",
    "keyField": "ItemId"
  },
  "fields": [
    { "key": "HdwNo",          "caption": "Hardware Number", "order": 1 },
    { "key": "HdwVendorlink", "caption": "Vendor Link", "order": 11, "type": "l"},
    { "key": "HdwImage",      "caption": "Image", "order": 13, "type": "image",
  ]
}

```

query property	Description
table	Table or view to query (<code>schema.table</code> or <code>[bracketed]</code> identifiers only — validated for safety).
keyColumn	Column in <code>table</code> used in the <code>WHERE</code> clause.
keyField	Property name on the tag object (case-sensitive!) whose value is passed as the lookup key.

Behavior:

- Runs `SELECT * FROM {table} WHERE {keyColumn} = @key` once per node selection (the key value is always parameterized — safe from injection).
- All returned columns become available as `key` values in `fields`, **except** columns that collide with an existing tag property name (tag properties win).
- If `table`/`keyColumn` aren't valid simple identifiers, or `keyField` doesn't match a real property, the query is silently skipped — fields fall back to "(none)" rather than breaking the pane.

Common mistake: `keyField` is case-sensitive and must match the C# property exactly (e.g. `DashId`, not `DashID`). If the query never seems to run, check this first.

5. Headings, separators, and collapsible sections

These entries have no `key` and don't pull any data — they're layout-only.

Separator (horizontal line)

```
json
{ "type": "separator", "order": 5 }
```

Heading

```
json
{ "type": "heading", "caption": "Vendor Information", "order": 6 }
```

- `caption` supports `{FieldName}` placeholders, same as `title`.

Collapsible section

```
json
{
  "type": "heading",
  "caption": "Additional Details",
  "order": 10,
  "collapsible": true,
  "collapsed": true
}
```

- `collapsible: true` adds a click-to-toggle arrow (▼ / ►).
- `collapsed: true` starts the section closed (omit or set `false` to start open).
- The section automatically includes every field row **after** this heading, up until the next `heading`/`separator` entry or the end of the `fields` array — no explicit "end" marker needed.

6. Full example

```
json
```

```

"HdwItemTag": {
  "title": "Hardware Item",
  "query": {
    "table": "dbo.Proj_HdwCompile",
    "keyColumn": "ID",
    "keyField": "ItemId"
  },
  "fields": [
    { "key": "HdwNo", "caption": "Hardware Number", "order": 1 },
    { "key": "HdwDesc", "caption": "Description", "order": 2 },
    { "key": "TotalDashQty", "caption": "Total Qty Used (All Dashes)", "order": 3 },

    { "type": "separator", "order": 5 },

    { "type": "heading", "caption": "Vendor Information", "order": 6, "collapse": true },
    { "key": "HdwVendorNum", "caption": "Vendor Part Number", "order": 7 },
    { "key": "HdwVendorlink", "caption": "Vendor Link", "order": 8, "type": "link", "linkText": true },

    { "type": "separator", "order": 9 },

    { "type": "heading", "caption": "Additional Details", "order": 10, "collapse": true },
    { "key": "HdwNotes", "caption": "Notes", "order": 11 },
    { "key": "HdwImage", "caption": "Image", "order": 12, "type": "image", "imageMimeType": "image/jpeg", "maxWidth": 100, "maxHeight": 100 },
    { "key": "GroupId", "caption": "Group ID", "order": 13 },
    { "key": "ItemId", "caption": "Item ID", "order": 14 }
  ]
}

```

7. Quick checklist when adding a new field

1. Does the property already exist on the tag class? → just add a `fields` entry with that `key`.
2. Need data not on the tag? → add/extend the `query` block:
 - Confirm `table` exists and has the column.
 - Confirm `keyColumn` matches a tag property's value (via `keyField`).
 - Double-check `keyField` casing against the C# class.
3. Pick a `type`: `text` (default), `link`, or `image` — set extra options (`linkText`, `imageMimeType`, `maxWidth`/`maxHeight`) as needed.
4. Set `order` to position it; use decimals if inserting between existing rows.

5. Save the file — no rebuild needed.